

Multi-Agent Embodied Architecture (multi_brains)

Multi-Agent Embodied Architecture (multi_brains)

1. Course Content
2. Introduction to multi_brains
 - 3.1 Advantages of Dual-Model Inference
 - 3.1.1 Decoupling of Task Decision and Action Conversion
 - 3.1.2 Improved Success Rate for Long-Process Tasks
 - 3.2 Decision-Making Layer AI
 - 3.3 Execution Layer AI
4. Task Cycle
5. Historical Context
6. Map Mapping
7. Action Function Library

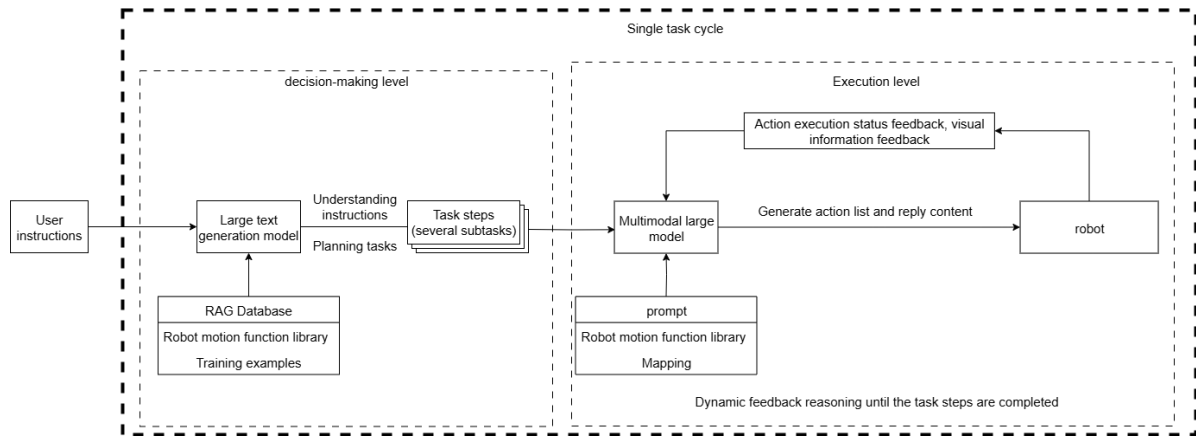
1. Course Content

Understanding the multi_brains multi-agent embodied architecture framework of ROSMASTER-M3 Pro

2. Introduction to multi_brains

- multi_brains is a self-developed embodied intelligence framework by Yabao Intelligent, adapted for general-purpose AI large models. It avoids the limitations of VLA models and end-to-end models, which require dedicated hardware, local computing power, and long data collection and training times. By simply connecting to general-purpose AI models from cloud model providers, the edge-side robot can achieve powerful embodied intelligence.
- multi_brains expands training examples through a RAG knowledge base, allowing the robot to quickly adapt to different scenarios and tasks. It can be deployed on most small main control devices, offloading the massive model inference to the cloud or a local server.
- The core idea of multi_brains is to decouple different stages of a task through multiple AI models. It is a comprehensive improvement on the first-generation self-developed dual-model inference framework. Its main improvements include:
- Smoother recording experience with inertial end-of-speech detection and adjustable end-of-speech detection duration.
- More flexible decision-making mechanism, adding task routing functionality. The AI agent can independently choose between dual-model and single-model inference based on task difficulty, giving the AI greater autonomy and a better user experience.
- Easy-to-understand visualized architecture process. multi_brains uses Dify to build a visualized intelligent agent, allowing real-time observation of the AI agent's operation and data flow during debugging, providing users with a more intuitive understanding of the dual-model inference process.
- Fully localized RAG knowledge base, eliminating concerns about data sensitivity.
- Seamless integration with most global AI models. Through Dify's model provider plugin, hundreds of AI models worldwide can be quickly and seamlessly integrated (local models can also be integrated if needed).

- Traceable history. By querying Dify's backend access history, users can view all task instructions and the robot's execution steps.
- Personalized response voice. Users can customize random response voices. The built-in speech synthesis commands are easy to configure and use.
- Comprehensive testing toolbox for quickly verifying core functional modules.
- Simpler setup steps, simplifying most operations and providing shortcut commands. ## 3. Dual-Model Inference Architecture
- The design of **dual-model inference** and **dynamic feedback inference** provides stronger system robustness compared to a single-model architecture.



3.1 Advantages of Dual-Model Inference

3.1.1 Decoupling of Task Decision and Action Conversion

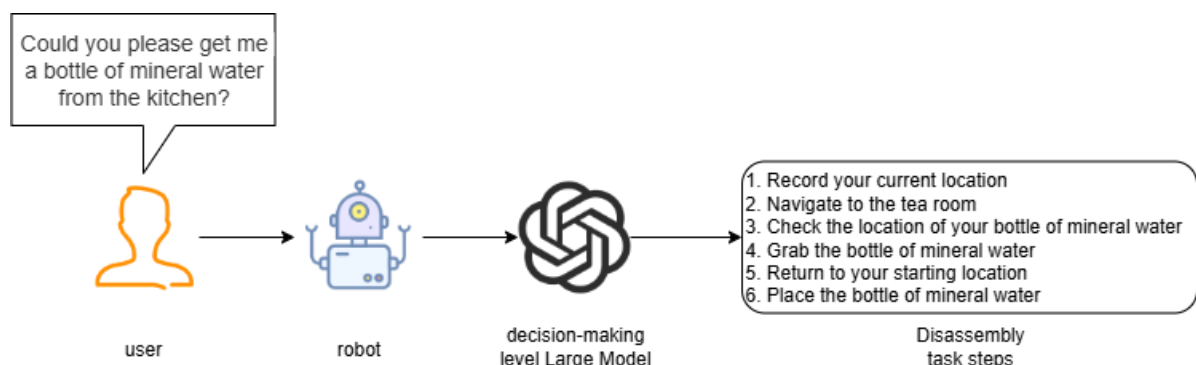
- The decision-making layer AI focuses on thinking, planning, and breaking down task instructions.
- The execution layer AI focuses on chat responses and converting task steps into JSON text.

3.1.2 Improved Success Rate for Long-Process Tasks

When using a single-model approach for inference, it requires simultaneous "natural language understanding + environmental perception + process decomposition + action function output," which is prone to **modal interference** (language ambiguity or overly long instructions leading to misinterpretation). The dual-model approach processes in stages, allowing the decision-making layer to focus on task planning and the execution layer to focus on action execution.

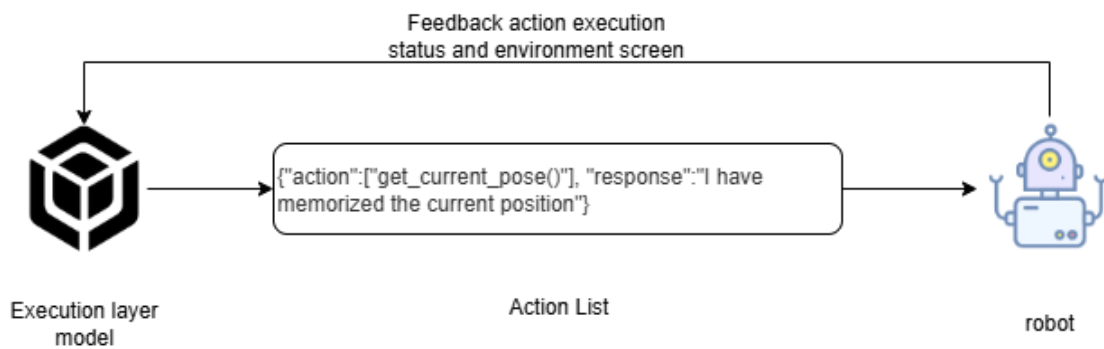
3.2 Decision-Making Layer AI

- The decision-making layer large language model is primarily responsible for task planning. It can understand complex human instructions and break them down into specific task steps. For example, when receiving the instruction "Can you get me a bottle of mineral water from the kitchen?", the large language model will break it down into several tasks, as shown in the figure below.



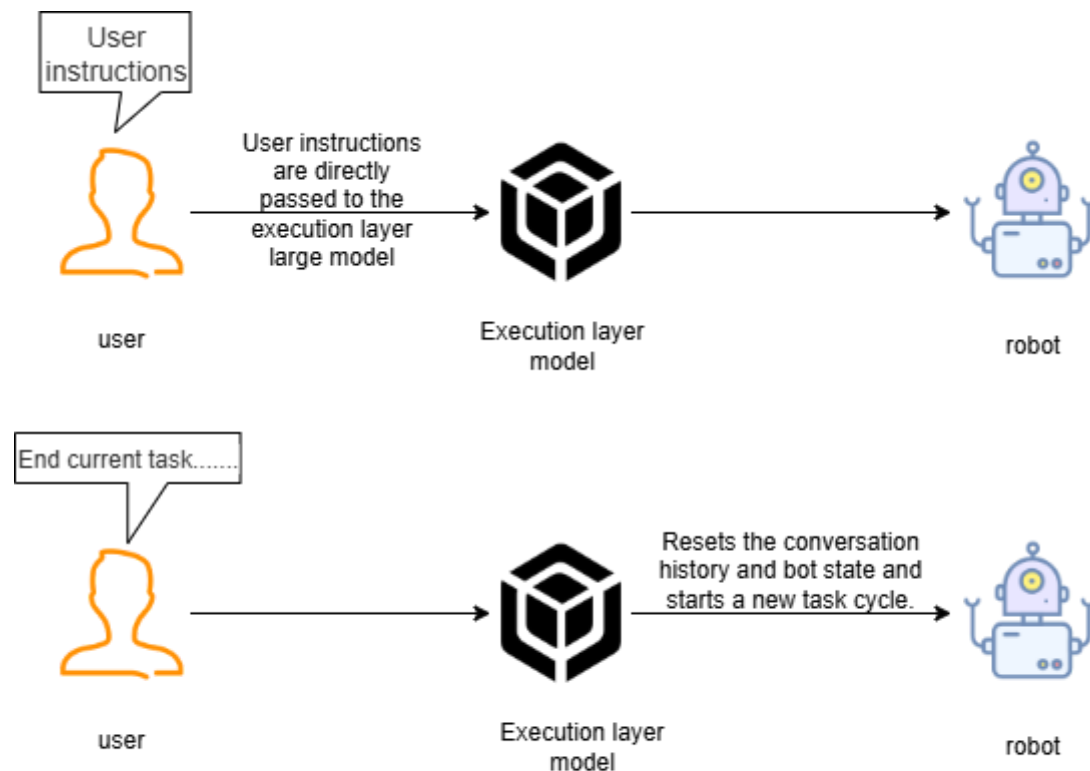
3.3 Execution Layer AI

- The execution layer large language model is primarily responsible for chat responses and converting task steps into JSON text. It continuously receives feedback (success/failure) and images from the robot's execution of actions, and provides instructions for the next action based on the success or failure of the action execution.
- The execution layer large language model acts as a supervisor, continuously monitoring the robot's progress in executing task steps and thinking and judging the next action to be performed based on the feedback from the robot's actions and environmental information, until the task is successfully completed or terminated prematurely due to special circumstances.



4. Task Cycle

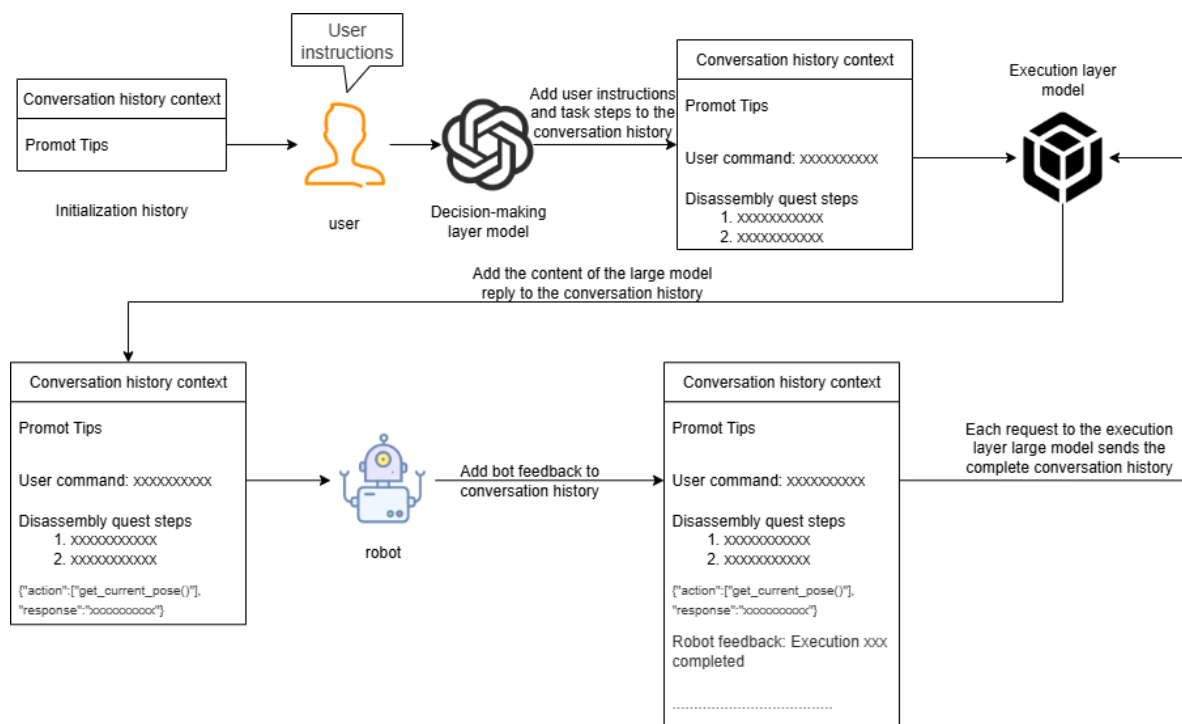
- After the robot is awakened, a task cycle begins. All user commands and robot feedback are included in the AI model's historical context, acting as the robot's temporary memory, allowing it to "remember" what happened before.
- When the user requests to end the current task and have the robot rest, the robot resets the AI model's historical context, effectively causing the robot to "forget" what happened before.



The waiting state of the robot after completing the task

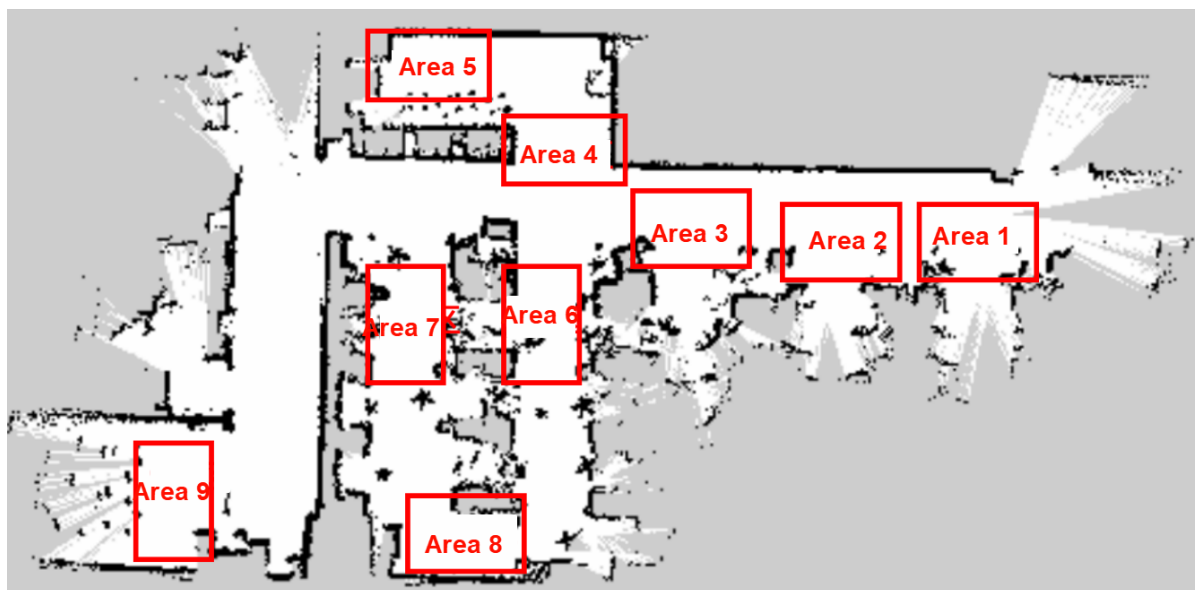
5. Historical Context

- The robot's conversation history is stored in Dify's AI application variables, with a default setting of a maximum conversation memory of 50 turns.



6. Map Mapping

- The robot uses a grid map for navigation. If the robot needs to understand location areas in the real world, a mapping relationship needs to be established between the grid map and the real-world environment areas. This relationship is called **map mapping**. - Let's assume we use a robot in a factory environment to generate a **grid map** using SLAM mapping. In the real factory, there are several manually defined areas, as shown in the image below:



We establish a one-to-one correspondence between these real-world areas and letter symbols:

A: "Area 1", B: "Area 2", C: "Area 3", D: "Area 4", E: "Area 5", F: "Area 6", G: "Area 7", H: "Area 8", I: "Area 9"

Then, we write the map coordinates for the letter symbols in a YAML file, for example:

```

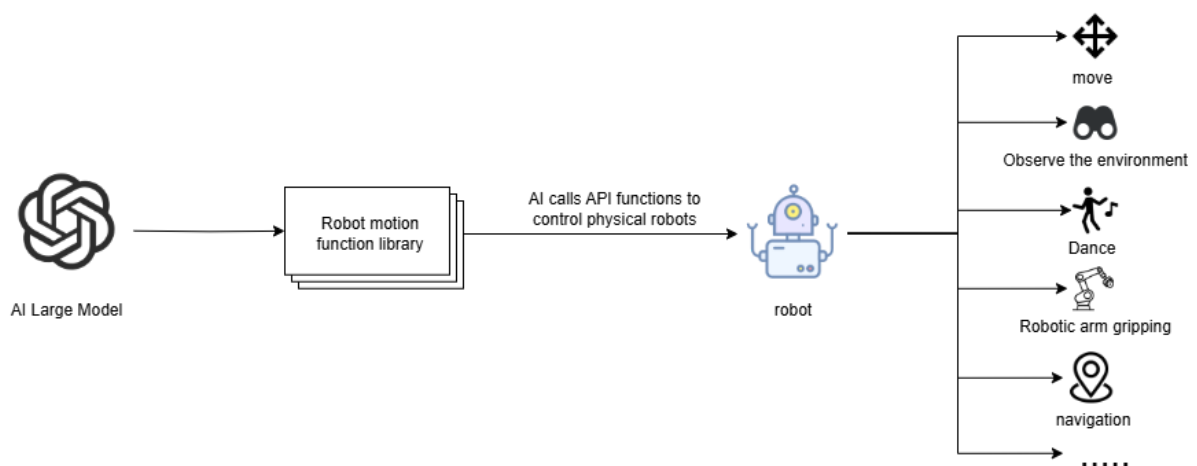
A:
name: 'Area 1'
position:
  x: 4.4034953117370605
  y: 0.4879316985607147
orientation:
  x: 0.0
  y: 0.0
  z: 0.701498621044694
  w: 0.7126708108744126

```

When we want the robot to go to a specific real-world area, we simply have the large language model convert the area name into the corresponding letter symbol, allowing the robot to understand the location in the real-world environment.

7. Action Function Library

- The API functions in the robot action function library are the bridge for the large language model to control the robot and interact with the real world.
- These API functions define the minimum actions that the physical robot can perform in the physical world.
- The AI large language model sends the required actions to the robot as JSON text, and the robot parses the JSON text to execute the corresponding actions.



All the minimum action functions and their corresponding functionalities are shown in the table below:

Function Name	Parameters	Functionality	Calling Command
move_left(x, angular_speed)	x: rotation angle, angular_speed: angular velocity	Rotate left by x degrees	Rotate left by x degrees
move_right(x, angular_speed)	x: rotation angle, angular_speed: angular velocity	Rotate right by x degrees	Rotate right by x degrees
dance()	-	Trigger the robot's preset dance movements	Dance, perform a dance, etc.
set_cmdvel(linear_x, linear_y, angular_z, duration)	linear_x: x-axis linear velocity, linear_y: y-axis linear velocity, angular_z: z-axis angular velocity, duration: topic publishing duration	Control the robot's chassis movement by setting linear and angular velocities	Move forward, backward, left, right
navigation(x)	x: the symbol corresponding to the target point in the map mapping	Control the robot to navigate to the target point	Navigate to a certain location
navigation(zero)	zero: the recorded coordinate point	Navigate back to the previously recorded coordinate point	Return to the origin, starting position
get_current_pose()	-	Record the current coordinates in the global map to the zero parameter	Remember the current position
arm_up()	-	Move the robotic arm upwards	Robotic arm up
arm_down()	-	Move the robotic arm downwards	Robotic arm down
arm_nod()	-	Robotic arm nods	Nod
arm_shake()	-	Robotic arm shakes its head	Shake head
arm_applaud()	-	Robotic arm applauds	Applaud

Function Name	Parameters	Functionality	Calling Command
grasp_obj(x1, y1, x2, y2)	(x1, y1, x2, y2) coordinates of the top-left and bottom-right points of the bounding box of the target object	Robotic arm grasps an object	Grasp xxx
putdown()	-		
apriltag_sort(x)	x: machine code number	Sort out the machine code with the specified number	Sort out machine code number x
track(x1, y1, x2, y2)	(x1, y1, x2, y2) Coordinates of the top-left and bottom-right points of the bounding box of the target object	Track the specified object within the field of view	Track xx
apriltag_remove_higher(x)	x: height	Remove machine codes at the specified height	Remove machine codes higher than x centimeters
color_remove_higher(color, target_high)	color: color target_high: height	Remove color blocks at the specified height	Remove color blocks of color with height greater than target_high centimeters
seewhat()	-	Take an image of the robot's current view and upload it to the execution layer large model	Observe the environment
follow_line_clear()	-	Move along the patrol line and clear machine code obstacles on the path	Start line clearing

Function Name	Parameters	Functionality	Calling Command
finish_dialogue()	-	Reset historical context and start a new task cycle	-
finish()	-	Automatically called after the robot completes the task, used to stop providing status feedback to the execution layer model	-